

Shared Execution-Clock Drifting Policy for Dynamic Precision Manipulation

Zhenchen Dong^{1,*}, Qingran Wu^{2,*}, Jinna Fu³, Jiaming Wu⁴
Fulin Chen⁵, Hongyu Yu^{1,†}, Yide Liu^{3,†}

Abstract—Manipulation under time constraints requires both accurate actions and an execution rhythm that matches the evolving scene. This becomes critical when a robot must intercept moving objects or complete a sequence of adjustments before a deadline. Although one-step policies reduce generation cost, their directly predicted action sequences leave temporal allocation implicit. We propose Shared Execution-Clock Drifting (SECD), which makes execution rhythm an explicit part of one-step action generation. Conditioned on an observation and a latent sample, the policy jointly predicts a progress-indexed action curve and a shared monotone clock that maps fixed control times to locations on the curve. Demonstration-derived alignment anchors this decomposition, which is trained jointly through drifting on the decoded actions. The resulting policy retains a fixed-rate control interface and requires one network evaluation. We evaluate SECD across four real-robot tasks with inference on NVIDIA Thor. Across 300 trials, it achieves 77.00% task-averaged success and outperforms the evaluated one-step baselines on every task, including 91% success in cup retrieval from a 16 m/min conveyor and 54% in restoring and folding a crumpled shirt within 90 s. A fixed-clock variant reaches 79% on the same conveyor protocol. Complementary state-based RoboMimic experiments, including cross-seed ablations on Transport and Square, further support the joint design of the temporal representation and demonstration alignment. Project page: <https://secd-anonymous-ewn.pages.dev/>.

I. INTRODUCTION

Conveyor grasping requires reaching a cup before interception becomes impossible, while closely spaced arrivals leave little time between retrievals. Restoring a crumpled shirt must likewise leave time for the remaining folds. Successful manipulation therefore depends on generated actions and the time assigned to their progression.

Our real-robot observations motivate studying these requirements together. Stationary tasks expose inaccurate placements and missed sleeve grasps. Conveyor retrieval adds missed interception opportunities, while crumpled-shirt recovery can exhaust the 90 s budget through ineffective adjustments. Differences among one-step conveyor policies further motivate learning execution rhythm alongside action quality, without isolating timing as every failure’s cause.

Action-chunk policies jointly predict time-indexed commands [1], [2], learning progression and timing implicitly

in the same outputs. They can express adaptive behavior, but expose no separate timing variable. Distillation and one-step generative methods reduce generation cost [3]–[6], and drifting supports whole-chunk generation [7], [8]. We ask: *can explicitly learning how execution time maps to action progress improve a one-step manipulation policy?*

We propose **Shared Execution-Clock Drifting (SECD)**, which explicitly learns execution-time allocation by jointly predicting a progress-indexed action curve and a monotone clock. Both depend on the observation and latent sample. The clock assigns fixed command times to curve locations, where a shared decoder reads the complete action vector. Controller frequency and prediction horizon remain fixed.

Different curve–clock pairs can decode to the same actions. Demonstration-derived alignment addresses this ambiguity by defining progress from normalized action changes and recording its time correspondence. These targets anchor the decomposition without event annotations. Drifting trains the decoded action distribution through both outputs, coupling progression and temporal allocation.

We evaluate four real-robot tasks with inference on NVIDIA Thor, including 90 s crumpled-shirt folding and conveyor retrieval. A fixed-clock conveyor comparison tests the complete representation, while Transport and Square ablations examine its interaction with alignment training.

Our contributions are:

- 1) **Joint learning of action progression and temporal allocation.** We introduce a one-step generator whose explicit execution clock is anchored by demonstration alignment and trained jointly with the progress curve through decoded-action drifting. The architecture and alignment objective together define the proposed temporal representation.
- 2) **Evaluation under real execution constraints.** Across four tasks with inference on NVIDIA Thor, SECD attains 77.00% task-averaged success. The conveyor comparison improves from 79% to 91% over a fixed clock, while simulation controls support the joint representation and alignment design.

II. RELATED WORK

A. Generative Visuomotor Policies

Diffusion Policy [1] models conditional action sequences for receding-horizon execution. Rectified flow [9] provides a transport-based formulation, while consistency policies [3], one-step diffusion distillation [4], shortcut models [10], and

*Equal contribution. †Corresponding authors: Hongyu Yu and Yide Liu.

¹The Hong Kong Polytechnic University.

²University of California San Diego.

³Zhejiang University.

⁴Université Paris-Saclay.

⁵Shanghai University of Engineering Science.

Correspondence: hongyu.yu@polyu.edu.hk;
yide.liu@zju.edu.cn.

Shared Execution-Clock Drifting (SECD)

One-Step Generation with Explicit Execution-Time Allocation

Learning Action and Rhythm Together!

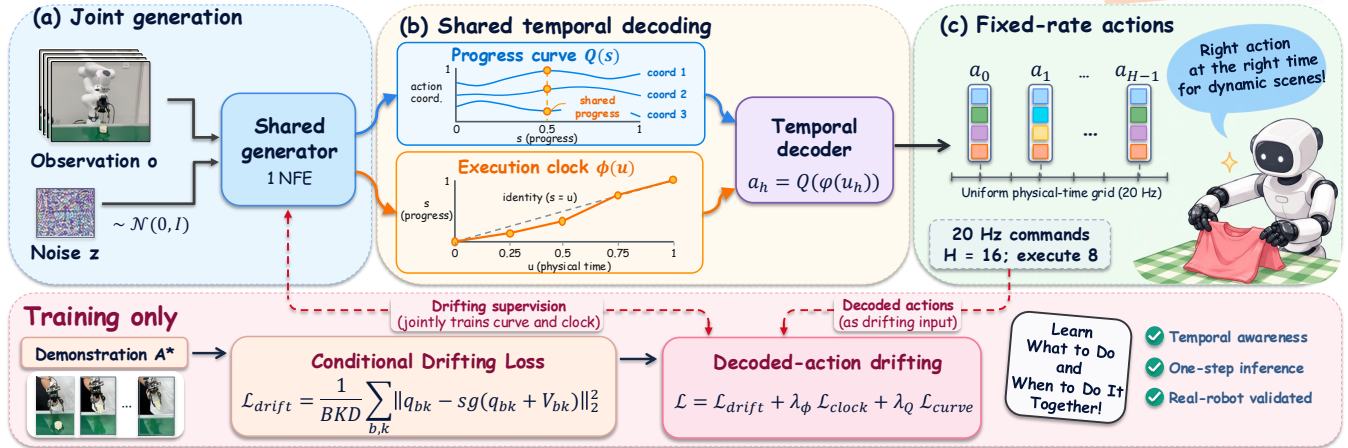


Fig. 1. **Shared Execution-Clock Drifting.** A joint generator predicts a progress curve and a monotone clock, which decode to fixed-rate actions in one network evaluation. Demonstration alignment and decoded-action drifting jointly train both outputs. Curves and clock profiles are schematic.

mean-flow policies [5], [6] reduce sampling cost. Hybrid-Flow [11] combines a global MeanFlow proposal with re-noising and local refinement in two network evaluations. These methods motivate efficient action generation. SECD investigates a progress-based output representation with an explicitly generated execution rhythm.

B. Drifting for Action Generation

Drifting trains a generator through attraction–repulsion targets and supports feature-space training [7]. DBPO [8] combines a one-step drifting policy with online policy optimization. DriftingVLA [12] introduces per-dimension temporal drifting and retains joint generation through a shared network. IDP [13] addresses sparse conditional expert samples through observation-related expert geometry. Our proposed modification concerns the progress curve, monotone clock, and temporal decoder. The drifting estimator, whole-chunk interactions, and one-step sampling remain inherited components.

C. Structured Actions and Temporal Warping

Movement primitives [14] and waypoint extraction [15] establish precedents for phase-based and compact motion representations. Spline Policy [16] predicts continuously decodable spline parameters and also constructs a state-dependent execution field. TimewarpVAE [17] jointly learns spatial representations and monotone time warps from full trajectories, with regularization addressing timing degeneracy. Thus, curve–clock decomposition and differentiable temporal decoding have precedents. SECD predicts both components from the current observation and latent sample at each policy query, using demonstration alignment and decoded-action drifting to train their joint distribution.

Other methods address different temporal decisions. PACE [18] selects how many predicted actions to execute before replanning. SPECTRA [19] regulates phase speed

under joint-motion constraints while retaining spectral coefficients. ISR [20] resamples demonstrations offline. SECD learns an intra-chunk time-to-progress map while keeping command frequency, prediction horizon, and executed-prefix length fixed. Its contribution concerns the generator parameterization and alignment objective.

III. SHARED-CLOCK ACTION GENERATION

We make execution-time allocation an explicit learning variable through three connected design choices (Fig. 1). Temporal allocation should be observation-conditioned, which motivates a generated clock. Progress should retain its order while its allocation changes, which motivates monotone decoding. The curve and clock need consistent training meanings, which motivates demonstration alignment. Drifting then supplies a joint training signal through the decoded actions. Implementation settings appear in Sec. IV.

A. A Joint Curve and Clock Generator

Let o denote the observation history, which contains state inputs in simulation and image-based observations with proprioception on the robot. The policy predicts H future action commands separated by Δt . Normalized execution time is $u_h = h/(H-1)$ for $h = 0, \dots, H-1$, with a physical horizon $T = (H-1)\Delta t$.

A progress variable $s \in [0, 1]$ indexes the generated action curve. It is local to a predicted chunk and is not an estimate of whole-task completion. An observation encoder and joint action head produce curve coefficients and clock logits:

$$c = E_\theta(o), \quad (C, v) = F_\theta(c, z), \quad z \sim \mathcal{N}(0, I). \quad (1)$$

Here $C \in \mathbb{R}^{L \times d_a}$ specifies a curve through a fixed piecewise-linear basis $b(s) \in \mathbb{R}^{d_a}$, which is differentiable away from its knots:

$$Q(s) = b(s)^\top C \in \mathbb{R}^{d_a}. \quad (2)$$

The curve and decoded commands use normalized action coordinates, which are unnormalized before execution. Clock logits $v \in \mathbb{R}^{H-1}$ allocate progress between command times. Both outputs depend on the same observation and latent sample within one policy call.

B. A Monotone Execution Clock

The clock maps normalized execution time to progress. We define positive increments with a lower bound ϵ , where $0 < \epsilon < 1/(H-1)$:

$$w_h = \epsilon + (1 - (H-1)\epsilon) \frac{\exp(v_h)}{\sum_{j=0}^{H-2} \exp(v_j)}. \quad (3)$$

With $\phi_0 = 0$, the clock nodes are

$$\phi_h = \sum_{j=0}^{h-1} w_j, \quad h = 1, \dots, H-1. \quad (4)$$

Thus $\phi_{H-1} = 1$ and $\phi_{h+1} > \phi_h$. Piecewise-linear interpolation between (u_h, ϕ_h) defines $\phi(u)$ throughout the horizon. Monotonicity preserves the order of points along Q ; it does not impose a task-specific order of operations.

The decoder evaluates

$$\hat{a}_h = Q(\phi_h), \quad h = 0, \dots, H-1. \quad (5)$$

The scalar ϕ_h is shared across action coordinates. Larger increments traverse more progress between commands, while smaller increments allocate more time to a curve portion. Controller frequency and prediction horizon remain fixed. In bimanual tasks, different coordinate functions can move or remain constant at the same progress, so sharing does not require equal velocities or simultaneous actions. The structural change is explicit progress-to-time factorization.

C. Anchoring the Decomposition with Demonstrations

Without a convention, the factorization is not identifiable. For any strictly increasing bijection $\psi : [0, 1] \rightarrow [0, 1]$, the two pairs

$$(Q, \phi) \quad \text{and} \quad (Q \circ \psi^{-1}, \psi \circ \phi) \quad (6)$$

produce the same continuous action sequence. A learned clock therefore needs an anchor if its values are to have a consistent interpretation across demonstrations.

We propose a training-only progress convention based on accumulated changes in normalized action commands. Given a demonstrated chunk $A^* = [a_0^*, \dots, a_{H-1}^*]$ and fixed dataset normalizer $\mathcal{N}$, define

$$r_h = \|W(\mathcal{N}(a_{h+1}^*) - \mathcal{N}(a_h^*))\|_2, \quad (7)$$

where $W = I$ after dataset normalization in our experiments. This convention measures normalized command changes, not physical speed, energy, or semantic task progress.

For a positive floor η , the target clock increments are

$$w_h^* = \epsilon + (1 - (H-1)\epsilon) \frac{r_h + \eta}{\sum_{j=0}^{H-2} (r_j + \eta)}. \quad (8)$$

Cumulative sums define ϕ_h^* . Interpolating $(\phi_h^*, \mathcal{N}(a_h^*))$ and sampling at fixed progress locations gives the curve target. The positive floor makes progress well-defined when action changes vanish. This convention anchors targets without proving a unique learned factorization. Nearly constant curve portions represent holds. Temporal allocation alone cannot repair an unsuitable action sequence.

D. Training through Temporal Decoding

Let $\hat{A}_{bk}$ be the decoded action sequence for observation b and latent sample k . We apply the established conditional drifting estimator [7], [8] to these normalized decoded sequences. Writing q_{bk} for its scaled action coordinates and V_{bk} for its detached field, the inherited frozen-target loss is

$$\mathcal{L}_{\text{drift}} = \frac{1}{BKD} \sum_{b,k} \|q_{bk} - \text{sg}(q_{bk} + V_{bk})\|_2^2, \quad (9)$$

where $D = Hd_a$. Candidate sampling, attraction–repulsion interactions, and normalization follow the cited estimator. They are not introduced as new components of SECD.

The proposed auxiliary terms anchor the clock and curve:

$$\mathcal{L}_{\text{clock}} = \frac{1}{H-2} \sum_{h=1}^{H-2} (\phi_h - \phi_h^*)^2, \quad (10)$$

$$\mathcal{L}_{\text{curve}} = \frac{1}{Jd_a} \sum_{j=1}^J \|Q(s_j) - Q^*(s_j)\|_2^2, \quad (11)$$

where $H > 2$ and the s_j are fixed progress samples. For $H = 2$, the clock is fixed by its endpoints and the clock loss is omitted. These terms are averaged across the minibatch and generated samples. The complete training objective is

$$\mathcal{L} = \mathcal{L}_{\text{drift}} + \lambda_\phi \mathcal{L}_{\text{clock}} + \lambda_Q \mathcal{L}_{\text{curve}}. \quad (12)$$

Pointwise curve and clock supervision can reduce diversity across valid alternatives. The weights and their controls must therefore be reported rather than assuming that the factorization preserves multimodality without qualification.

E. Principle: Drifting through a Shared Execution Clock

The DBP backbone of DBPO [8] directly parameterizes time-indexed action output. Our composite map $\hat{a}_h = Q_C(\phi_v(u_h))$ exposes progression and temporal allocation as intermediate variables. Both policies return $H \times d_a$ commands. At $L = H$, a uniform clock gives $\hat{A} = C$, so the representation includes arbitrary discrete command sequences. The proposed change is a parameterization and training bias, without a claim of a larger action space.

Curve and timing variations. Suppress the observation and sample indices and consider a differentiable point of the decoder. Its first-order action variation is

$$\delta \hat{a}_h = b(\phi_h)^\top \delta C + Q'_C(\phi_h) \delta \phi_h + o(\|\delta C\| + \|\delta \phi\|). \quad (13)$$

For fixed C , a timing variation moves the action vector along the local tangent $Q'_C(\phi_h)$. Curve-coefficient variations can have both tangential and normal components, so Eq. (13) is not an orthogonal decomposition. The overlap motivates the target convention in Eq. (8).

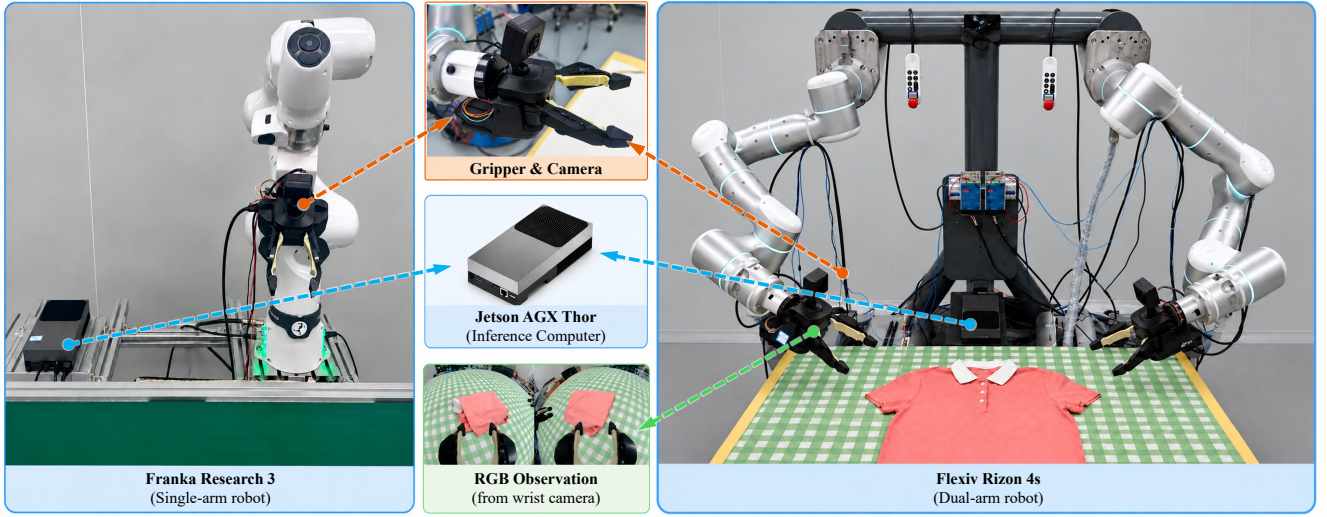


Fig. 2. **Real-robot hardware setup.** A Franka Research 3 arm performs conveyor retrieval (left). The Flexiv Rizon 4s setup (right) uses one arm for cup placement and both arms for shirt manipulation. The insets show the gripper and wrist camera, example RGB observations, and the NVIDIA Jetson AGX Thor computer on which all compared policies run inference.

The drift-induced clock signal. Let $\sigma > 0$ denote the detached scale used by the inherited estimator, so that $q = \text{vec}(\hat{A})/\sigma$. Write V_h for the corresponding d_a -dimensional block of the frozen field and $\kappa = 2/(BKD\sigma)$. For this sample’s contribution to Eq. (9), the partial derivatives are

$$\nabla_C \mathcal{L}_{\text{drift}} = -\kappa \sum_h b(\phi_h) V_h^T, \quad (14)$$

$$\frac{\partial \mathcal{L}_{\text{drift}}}{\partial v_j} = -\kappa \sum_h \underbrace{\langle Q'_C(\phi_h), V_h \rangle}_{\text{timing signal at } h} \frac{\partial \phi_h}{\partial v_j}. \quad (15)$$

These expressions follow by differentiating the prediction while holding the drift target and scale fixed. The inner product aggregates all action dimensions into a scalar timing signal. A field component orthogonal to $Q'_C(\phi_h)$ contributes no clock signal at that time, although it can still supervise the curve. On a constant segment, clock learning from decoded actions vanishes locally and clock supervision provides an additional signal.

Let $\pi_j = \text{softmax}(v)_j$ and $\alpha = 1 - (H - 1)\epsilon$. The clock Jacobian is explicit:

$$\frac{\partial \phi_h}{\partial v_j} = \alpha \pi_j \left(\mathbf{1}_{\{j < h\}} - \sum_{i=0}^{h-1} \pi_i \right). \quad (16)$$

Changing one logit redistributes progress through the normalized increments while retaining the clock endpoints. Thus Eq. (15) couples the action-space signal to a horizon-wide temporal allocation. It does not prescribe independent per-dimension timing.

These output derivatives describe how the inherited field trains the composite generator. Shared network parameters and auxiliary losses combine curve and clock signals, so parameter updates need not produce isolated time shifts or exact steps along V . The identities imply no convergence or task-success guarantee.

F. Implementation and One-Pass Deployment

Shared encoder and U-Net features feed jointly trained linear curve and clock heads. The curve head produces L progress knots, and clock outputs are linearly resampled to $H - 1$ logits. Zero initialization gives a uniform initial clock. Piecewise-linear decoding retains gradients through interpolation weights and adds no network evaluation. Mean policy inference time on NVIDIA Thor is 25 ms.

Decoding precedes unnormalization and robot-interface conversion. The 6D orientation representation requires valid-rotation conversion and degenerate-encoding handling. Simulation retains native incremental commands where applicable, so decoded curves need not correspond to preserved Cartesian paths after controller integration.

At deployment, one latent sample generates both outputs without expert-derived targets. The controller executes a short prefix and replans. Clock values reset per prediction and do not represent persistent task-wide phase.

IV. EXPERIMENTS

The evaluation asks: **Q1:** How does SECD compare with direct-output policies? **Q2:** How effective is it under real execution constraints on NVIDIA Thor? **Q3:** How do the temporal representation and alignment training contribute? Four real-robot tasks address Q1 and Q2. Conveyor, Transport, and Square ablations address Q3, alongside four-task simulation benchmarks.

A. Real-Robot Tasks and Data

Hardware. Figure 2 shows the Franka Research 3 conveyor setup and the Flexiv Rizon 4s setup, which uses one arm for cup placement and both arms for shirt manipulation. Wrist cameras provide RGB observations. All compared real-robot policies run inference on NVIDIA Jetson AGX Thor with common robot-side deployment settings within each task.

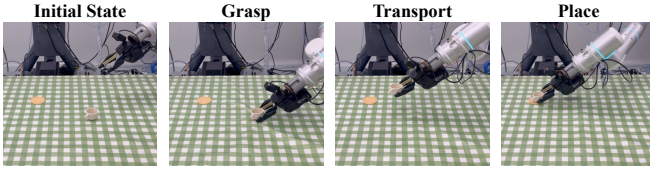


Fig. 3. **Cup Place: placing a cup on a coaster.** A single arm grasps and transports a stationary cup to a coaster whose position varies across trials.

Tasks. The four tasks expose different operational requirements:

- 1) **Cup Place (Fig. 3).** A single arm grasps a stationary cup and places it on a randomly positioned coaster. Success requires aligning the cup and coaster centers. Grasp acquisition and release placement are analyzed separately.
- 2) **Fold (Flat) (Fig. 4, lower row).** Two arms fold the right and left sleeves of a flat shirt, fold it in half, rotate it by 90° , and fold it in half again. Success is judged by the final configuration after both half-folds. Sleeve grasping tests local operation quality within a longer sequence.
- 3) **Fold (Crumpled) (Fig. 4, both rows).** A casually tossed shirt starts in a random crumpled configuration. The robot must first flatten it and then complete the same folding sequence within 90s. Exceeding the deadline is a failure. This task also tests recovery from unsuitable grasps, which a shared clock alone does not guarantee.
- 4) **Conveyor Cup (Fig. 5).** A single arm grasps a cup moving on a 16m/min conveyor and deposits it into a basket. The approach and grasp must fit a very short interception window. The task tests whether the generated progression and its execution rhythm fit target motion.

Demonstrations. Within each task, compared policies use the same dataset and observation-encoder architecture. The established datasets contain 2h for Cup Place, 10h each for Fold (Flat) and Fold (Crumpled), and 3h for Conveyor Cup. The proposed progress targets are derived from these existing action sequences, using the same normalizer as policy training. No additional event or task-phase annotation is assumed.

B. Comparisons and Implementation Protocol

Policy comparisons. DP [1] uses 16 inference evaluations. OneDP [4] distills a teacher configured for 16-step DDIM sampling into a one-evaluation student. Native Drifting [7] denotes our local direct-output policy. Its supplied implementation generates $K = 8$ candidates per observation, flattens each full horizon to Hd_a coordinates, and applies drifting with the paired expert sequence as the positive reference and generated candidates as repulsive references. It uses the same drifting-loss routine as SECD, with one-sample, one-evaluation deployment. SECD adds the curve-clock parameterization and alignment losses. All methods

share task datasets and encoder architecture.

Fixed-clock ablation. We additionally train and evaluate a conveyor variant with $\phi_h = h/(H - 1)$. It retains the backbone and curve head but removes the learned clock head and clock-alignment loss, and uses uniform-progress curve targets. At $L = H$, uniform decoding samples the curve coefficients directly. This control tests the full aligned temporal representation rather than the clock branch alone.

Configuration. The real-robot configuration uses two observation steps, 224×224 RGB inputs, a DINOv3 ViT-B/16 encoder [21], and a conditional 1D U-Net with widths [256, 512, 1024]. The horizon is $H = 16$, with 10 action coordinates per arm: relative position, 6D orientation, and a scalar gripper command.

Training configuration. The decoder uses piecewise-linear interpolation with $L = J = H$, $\epsilon = \eta = 10^{-4}$, $W = I$ in normalized coordinates, and $\lambda_\phi = \lambda_Q = 1$. Drifting uses $K = 8$ and temperatures $\{0.02, 0.05, 0.2\}$. The supplied configurations specify 200 epochs, batch size 192, seed 42, and 5% validation data. AdamW uses generator/encoder learning rates $10^{-4}/10^{-5}$, $\beta = (0.95, 0.999)$, weight decay 10^{-6} , cosine decay, and 2,000 warmup steps. EMA uses power 0.75 and maximum decay 0.9999. OneDP freezes its teacher-initialized encoder; the drifting policies train it jointly.

Temporal alignment and deployment. WebDataset training uniformly samples a 0–4-step offset, limited by available sample length. Images stay fixed while proprioception and actions shift together. Actions begin at the last proprioceptive timestep, and clock targets use the shifted action window. Relative poses retain the latest-image reference. All policies run on NVIDIA Thor at a command rate of 20 Hz ($\Delta t = 50$ ms), executing eight steps between replans (nominally 0.4 s) under common robot-side settings. Code and training configurations will be released.

C. Real-Robot Results and Operation Analysis

Overall performance. Table I reports 77.00% task-averaged success for SECD. Gains over Native Drifting are 12, 20, 28, and 35 percentage points in task order. SECD leads the evaluated one-step policies on every task and all policies on the two tasks with explicit timing constraints.

Statistical uncertainty. The two-sided 95% Wilson intervals for SECD are [88.8, 97.8]%, [54.2, 79.2]%, [40.4, 67.0]%, and [83.8, 95.2]% in table order. These quantify trial-level uncertainty, not training-seed variability. The following analysis discusses task-level failure modes illustrated in the figures.

Cup acquisition and placement. SECD reaches 95% success, close to DP’s 98% and above Native Drifting’s 83% and OneDP’s 71%. This basic task mainly requires reliable acquisition and accurate release. Baseline observations identified empty initial grasps with OneDP and releases too far from the coaster (Fig. 6). The shared-clock policy retains high precision, although the aggregate counts do not identify the causes of its five failures.

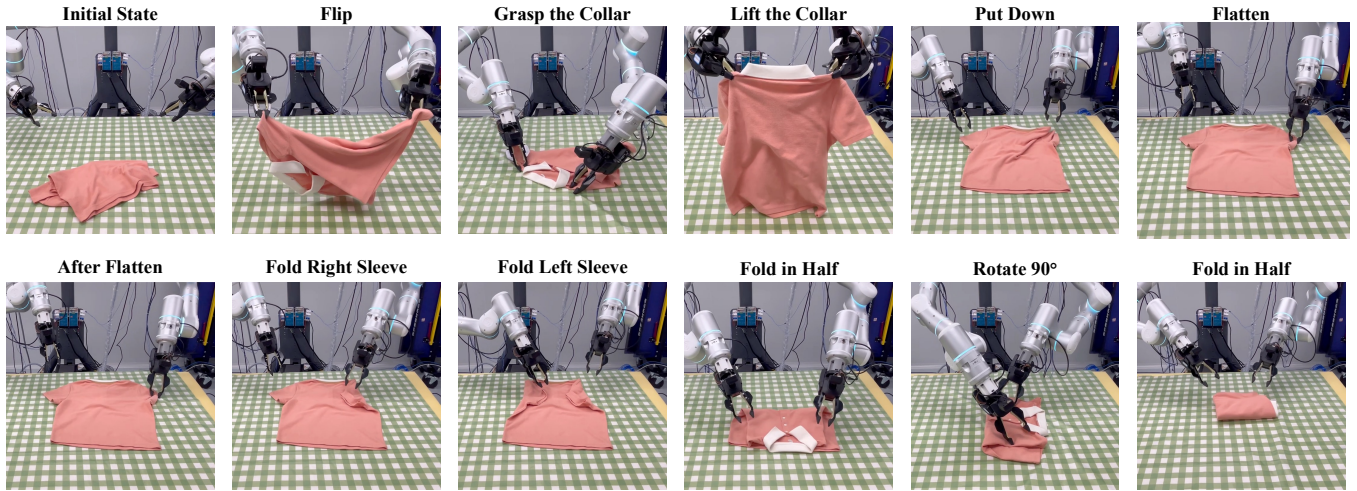


Fig. 4. **Shirt folding from flat and crumpled states.** The upper row shows recovery from a crumpled state through flipping, collar grasping and lifting, lowering, and flattening. The lower row shows right- and left-sleeve folding, a half-fold, a 90° rotation, and a second half-fold. Fold (Crumpled) requires the full sequence within 90 s; Fold (Flat) starts from the flattened state.

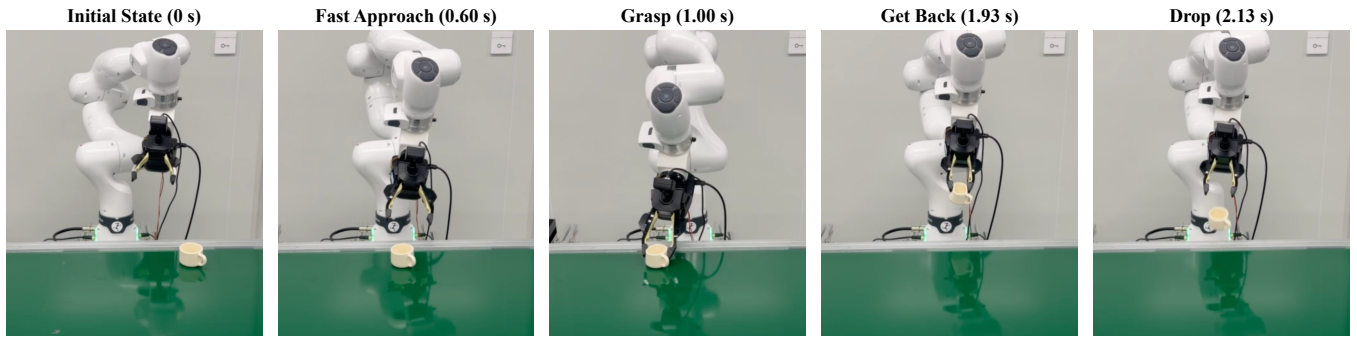


Fig. 5. **Conveyor Cup: retrieving a moving cup.** The robot approaches and grasps a cup on a conveyor moving at 16 m/min before withdrawing and releasing it. The timestamps indicate elapsed time in the illustrated execution sequence.

TABLE I

REAL-ROBOT SUCCESS (%) AND MEAN POLICY INFERENCE TIME ON NVIDIA THOR. EACH POLICY IS EVALUATED ON 100 CUP PLACE, 50 FOLD (FLAT), 50 FOLD (CRUMPLED), AND 100 CONVEYOR CUP TRIALS. AVG. IS THE UNWEIGHTED MEAN ACROSS TASKS. FOLD (CRUMPLED) REQUIRES FLATTENING AND FULL FOLDING WITHIN 90 s; CONVEYOR CUP USES A BELT SPEED OF 16 M/MIN. BOLD MARKS THE HIGHEST SUCCESS IN EACH COLUMN.

Method	NFE	Lat. (ms)	Cup Place	Fold (Flat)	Fold (Crumpled)	Conveyor Cup	Avg.
DP [1]	16	252	98.0	74.0	30.0	0.0	50.50
OneDP [4]	1	26	71.0	38.0	22.0	31.0	40.50
Native Drifting [7]	1	24	83.0	48.0	26.0	56.0	53.25
SECD (ours)	1	25	95.0	68.0	54.0	91.0	77.00

Sleeve grasping and folding. SECD completes 34/50 trials (68%), compared with 74% for DP, 48% for Native Drifting, and 38% for OneDP. This task emphasizes action quality because a missed sleeve grasp can prevent subsequent folds (Fig. 7). The 20-point gain over Native Drifting extends the advantage to multi-stage manipulation without a deadline, while DP’s remaining lead indicates room for improving fine manipulation.

Recovery within 90 s. SECD completes 27/50 trials (54%), exceeding DP’s 30%, Native Drifting’s 26%, and OneDP’s 22%. The ranking changes from flat-shirt folding,

where DP leads. Crumpled starts add variable recovery work and require flattening early enough to finish both folds. Observations showed that grasping multiple fabric layers can cause ineffective adjustment cycles. The result supports the complete policy under this combined demand for action quality and timely completion. Further mechanism analysis is discussed in Sec. V.

Moving-cup interception. SECD succeeds in 91/100 trials on the 16 m/min conveyor, compared with 56% for Native Drifting, 31% for OneDP, and 0% for DP. Its 35-point lead over Native Drifting is the largest task-wise gain. The

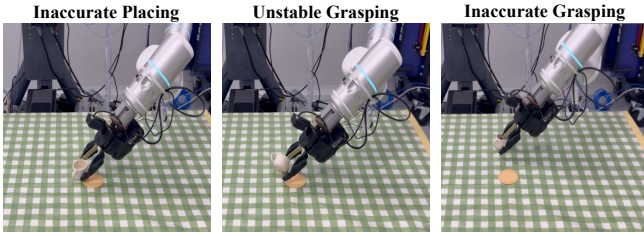


Fig. 6. **Cup grasping and placement failures.** The examples show inaccurate placement (left), unstable grasping (center), and inaccurate grasping (right).

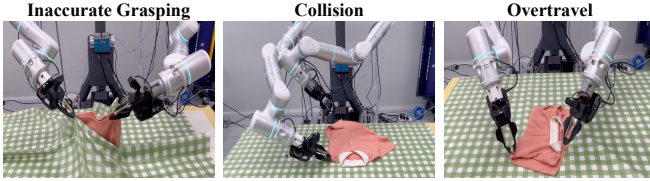


Fig. 7. **Failures during shirt manipulation.** The examples illustrate inaccurate grasping, collision, and overtravel that disrupt the manipulation sequence.



Fig. 8. **Missed conveyor grasp with DP (16 NFE).** The cup continues moving during the approach, and the gripper fails to intercept it.

approach and grasp must fit a brief interception opportunity, which the illustrated DP sequence misses (Fig. 8). The strong separation among one-step policies shows that a single network evaluation alone is insufficient. The fixed-clock ablation below further examines the temporal representation.

D. Fixed-Uniform-Clock Conveyor Ablation

Under the same data, training budget, and deployment settings, the separately trained fixed-clock variant retrieves 79/100 cups, compared with 91/100 for complete SECD, a 12-percentage-point gain. Both evaluations use the 16 m/min conveyor and deliberately include closely spaced cup pairs. Success is counted per cup, not per pair. The two-sided 95% Wilson intervals are $[70.0, 85.8]\%$ and $[83.8, 95.2]\%$, respectively.

Closely spaced arrivals. In observed sequences with two cups arriving in close succession, the fixed-clock policy failed to complete retrieval, while SECD completed such sequences. This is consistent with the benefit of adapting execution rhythm to successive interception opportunities. It does not establish a minimum arrival gap or a condition-specific success rate. The ablation supports the complete aligned temporal representation. The cup-level intervals do not model dependence between successive cups.

E. Complementary State-Based Simulation

State-based experiments use the RoboMimic proficient-human Lift, Can, Square, and Transport datasets [24], retaining the previous setup. The U-Net has widths $[256, 512, 1024]$, kernel size 5, eight normalization groups, and generation-index embedding dimension 256. It conditions on two observation steps, predicts 16 actions, and executes eight before replanning. Training uses seed 1 and batch size 100 for 2,000 epochs of 100 updates, AdamW at 10^{-4} with weight decay 10^{-6} , cosine decay with 500 warmup steps, and EMA power 0.75. Each evaluation uses 100 rollouts and runs every 50 epochs, with early termination on task success.

Table II reports best success and final-five-checkpoint averages. The task means for SECD are 0.9375 and 0.8900, displayed as 0.94 (0.89). Transport improves numerically from the cited Native Drifting result of 0.68 (0.58) to 0.78 (0.68). Square matches its 0.97 (0.90), while DP reports a final-five average of 0.92. CP denotes Consistency Policy, distinct from real-robot OneDP. Literature comparisons do not isolate architectural effects.

The decoder retains each benchmark’s action representation and output conversion. Decoding incremental commands need not preserve the accumulated physical path.

F. Representation and Alignment Ablations

Additional local Transport and Square ablations report results aggregated across training seeds, with 100 rollouts per checkpoint evaluation and otherwise unchanged settings. Table III compares direct-output drifting (A), the fixed-clock variant with action MSE (B), SECD with both alignment weights set to zero (C), and complete SECD (D). C and D share the same architecture and parameter count. At $L = H$, B’s uniform decoder and curve target reduce to direct action prediction and action MSE.

On Transport, action MSE improves B over A by 6/2 percentage points (Best/Final), whereas C changes A by +2/−2 points. Complete SECD exceeds the same-architecture C by 8/12 points and B by 4/8 points. On Square, D matches C’s best success and improves its Final by 2 points, while trailing B’s Final by 2 points. Thus, alignment benefits the learned-clock architecture on both tasks, while the additional gain over uniform-clock action regression depends on the task. These aggregated results neither isolate the two alignment losses nor quantify cross-seed dispersion.

V. LIMITATIONS AND FUTURE WORK

The results support the complete policy but do not identify which stages advance faster or slower. Future work will relate predicted clocks to observations and execution events. Transport and Square controls support alignment within the same architecture but do not isolate its two losses. Generalization to other tasks and per-seed variability analysis remain future work.

Progress and time admit equivalent parameterizations. Demonstration alignment anchors targets without establishing a unique representation. Its action-change metric depends

TABLE II

STATE-BASED ROBOMIMIC PH SUCCESS: BEST (FINAL-FIVE-CHECKPOINT AVERAGE); AVG.: FOUR-TASK MEAN. †: IDP TABLE 2 [13], AVERAGING CNN/TRANSFORMER BACKBONES, THREE SEEDS, AND 50 INITIAL CONDITIONS; NFE FOLLOWS THE SOURCE. NATIVE DRIFTING DENOTES ITS “NAIVE DRIFTING.” SECD: LOCAL U-NET, 100 ROLLOUTS/CHECKPOINT.

Method	NFE	Lift	Can	Square	Transport	Avg.
DP [1]†	100	1.00 (1.00)	1.00 (0.98)	0.96 (0.92)	0.73 (0.62)	0.92 (0.88)
Flow [22]†	9	1.00 (1.00)	1.00 (0.98)	0.95 (0.90)	0.74 (0.59)	0.92 (0.87)
MIP [23]†	2	1.00 (1.00)	1.00 (0.98)	0.96 (0.88)	0.71 (0.58)	0.92 (0.86)
CP [3]†	1	1.00 (1.00)	0.99 (0.96)	0.88 (0.82)	0.32 (0.24)	0.80 (0.76)
MP1 [6]†	1	1.00 (1.00)	1.00 (0.97)	0.96 (0.88)	0.52 (0.39)	0.87 (0.81)
Native Drifting [7]†	1	1.00 (1.00)	0.99 (0.96)	0.97 (0.90)	0.68 (0.58)	0.91 (0.86)
IDP [13]†	1	1.00 (1.00)	1.00 (0.97)	0.95 (0.88)	0.72 (0.58)	0.92 (0.86)
SECD (ours)	1	1.00 (1.00)	1.00 (0.98)	0.97 (0.90)	0.78 (0.68)	0.94 (0.89)

TABLE III

TRANSPORT AND SQUARE ABLATIONS: SUCCESS RATES AGGREGATED ACROSS TRAINING SEEDS. BEST AND FINAL DENOTE BEST-CHECKPOINT SUCCESS AND THE FINAL-FIVE-CHECKPOINT AVERAGE, RESPECTIVELY.

Variant	Transport		Square	
	Best	Final	Best	Final
A: Direct drifting	0.68	0.58	0.97	0.90
B: Uniform clock + MSE	0.74	0.60	0.96	0.92
C: SECD w/o alignment	0.70	0.56	0.97	0.88
D: Complete SECD	0.78	0.68	0.97	0.90

on normalization and does not measure semantic progress. Clock non-uniformity alone does not establish useful adaptation, which also depends on the curve. Basis resolution, positive increments, and auxiliary losses can limit fidelity or diversity. Inherited drifting does not resolve missing conditional expert modes.

A monotone clock preserves curve traversal order without guaranteeing dynamic feasibility. Controller interpolation, actuator dynamics, and observation delay affect execution. The local clock provides neither task-level recovery planning nor a deadline guarantee. Table II’s baseline rows are literature results, whereas Table III reports local controls. Conveyor evaluation covers one speed, and real-robot trial intervals do not quantify training-seed variability.

VI. CONCLUSION

Shared Execution-Clock Drifting explicitly learns temporal allocation through an aligned curve-clock parameterization. It achieves 77.00% task-averaged real-robot success on NVIDIA Thor, including 54% in deadline-constrained folding and 91% in conveyor retrieval. The conveyor and simulation ablations support the combined representation and alignment design. RoboMimic evaluation yields 93.75% task-averaged best success and 89.00% final-five-checkpoint average success.

REFERENCES

- [1] C. Chi *et al.*, “Diffusion policy: Visuomotor policy learning via action diffusion,” *Int. J. Robot. Res.*, vol. 44, no. 10–11, pp. 1684–1704, 2025.
- [2] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning fine-grained bimanual manipulation with low-cost hardware,” in *Proc. RSS*, 2023.
- [3] A. Prasad, K. Lin, J. Wu, L. Zhou, and J. Bohg, “Consistency policy: Accelerated visuomotor policies via consistency distillation,” in *Proc. RSS*, 2024.
- [4] Z. Wang *et al.*, “One-step diffusion policy: Fast visuomotor policies via diffusion distillation,” *arXiv:2410.21257*, 2024.
- [5] Z. Geng, M. Deng, X. Bai, J. Z. Kolter, and K. He, “Mean flows for one-step generative modeling,” in *Proc. NeurIPS*, vol. 38, 2025.
- [6] J. Sheng, Z. Wang, P. Li, and M. Liu, “MP1: MeanFlow tames policy learning in 1-step for robotic manipulation,” in *Proc. AAAI*, vol. 40, no. 22, 2026, pp. 18532–18539.
- [7] M. Deng, H. Li, T. Li, Y. Du, and K. He, “Generative modeling via drifting,” *arXiv:2602.04770*, 2026.
- [8] Y. Gao *et al.*, “Drift-based policy optimization: Native one-step policy learning for online robot control,” *arXiv:2604.03540*, 2026.
- [9] X. Liu, C. Gong, and Q. Liu, “Flow straight and fast: Learning to generate and transfer data with rectified flow,” in *Proc. ICLR*, 2023.
- [10] K. Frans, D. Hafner, S. Levine, and P. Abbeel, “One step diffusion via shortcut models,” in *Proc. ICLR*, 2025.
- [11] Z. Dong, J. Fu, J. Wu, S. Yu, F. Chen, and Y. Liu, “HybridFlow: A two-step generative policy for robotic manipulation,” *arXiv:2602.13718*, 2026.
- [12] Y. Gao *et al.*, “DriftingVLA: Native one-step vision-language-action generation via per-dimension temporal drifting,” *arXiv:2608.29749*, 2026.
- [13] Z. Yang *et al.*, “Implicit drifting policy: One-step action generation via conditional expert geometry,” *arXiv:2606.01098*, 2026.
- [14] A. J. Ijspeert, J. Nakanishi, H. Hoffmann, P. Pastor, and S. Schaal, “Dynamical movement primitives: Learning attractor models for motor behaviors,” *Neural Computation*, vol. 25, no. 2, pp. 328–373, 2013.
- [15] L. X. Shi, A. Sharma, T. Z. Zhao, and C. Finn, “Waypoint-based imitation learning for robotic manipulation,” in *Proc. CoRL*, 2023, pp. 2195–2209.
- [16] M. Tian, Y. Li, S. Liu, A. Ijspeert, and S. Calinon, “Spline policy: A structured representation for robot policies,” *arXiv:2606.07386*, 2026.
- [17] T. Rhodes and D. D. Lee, “TimewarpVAE: Simultaneous time-warping and representation learning of trajectories,” *arXiv:2310.16027*, 2023.
- [18] J. Nie *et al.*, “PACE: Phase-aware chunk execution for robot policies with action chunking,” *arXiv:2606.00537*, 2026.
- [19] B. Zhang, S. Liu, C. Ming, and A. Abdelrahman, “SPECTRA: Context-conditioned spectral movement primitives for robot skill generalization,” *arXiv:2607.06978*, 2026.
- [20] L. Yang *et al.*, “Improving robotic imitation learning via trajectory standardization,” *arXiv:2606.22907*, 2026.
- [21] O. Siméoni *et al.*, “DINOv3,” *arXiv:2508.10104*, 2025.
- [22] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, “Flow matching for generative modeling,” in *Proc. ICLR*, 2023.
- [23] C. Pan *et al.*, “Much ado about noising: Dispelling the myths of generative robotic control,” *arXiv:2512.01809*, 2025.
- [24] A. Mandlekar *et al.*, “What matters in learning from offline human demonstrations for robot manipulation,” in *Proc. CoRL*, PMLR, vol. 164, 2022, pp. 1678–1690.